

Kiste Phase 9 / v0.9.0 Spec

Library Fix, Static Plugin Architecture, Package Manager Vision, and Three-Way Split-Ready Repository Design

One-sentence spec: Kiste v0.9.0 stabilizes Kiste architecture by fixing the library/CLI boundary, introducing a static plugin system, defining module/package metadata and a lockfile, and designing the repo so kiste-core, kiste-py, and kiste-cli can become separate repos later without architectural surgery.

Release	Primary purpose
Phase 5	Python library + CLI wrapper, with CLI calling library logic.
Phase 8	Self-hosting, modular bootstrap, cloud readiness, and Argo CD preparation.
Phase 8.5	Release stabilization: tests, docs, security, packaging, and freeze checks.
Phase 8.5.1	Data/blob storage revision: metadata-first blob references and sample-only reads.
Phase 9 / v0.9.0	Architecture release: static plugins, package-manager foundation, and split-ready repo boundaries.

Phase 9 should not add major new product features. Its purpose is to make Kiste architecturally serious before the project grows further.

1. Phase 9 Goal

Phase 9 is an architecture-hardening release. The key output is a stable internal structure that can survive growth into many providers, cloud targets, data systems, and user interfaces.

- Fix the library/CLI boundary so CLI is only an interface, not the product core.
- Introduce static plugins first, with known metadata, dependencies, capabilities, and commands.
- Create module/package metadata and a lockfile as the foundation for a future package-manager-like ecosystem.
- Design the monorepo so it can later split into kiste-core, kiste-py, and kiste-cli.
- Avoid new major features until architecture tests pass.

Main rule:

No new feature until Kiste has a clean core, static plugins, stable APIs, a module manifest, and a lockfile.

2. What Phase 9 Is Not

- Not a new cloud provider phase.
- Not a new data provider phase.
- Not a SaaS or marketplace launch.
- Not dynamic plugin download from GitHub or unknown registries.
- Not automatic plugin update or remote code execution.
- Not a production deployment phase.

Allowed work: refactor, split internals, stabilize APIs, define plugin contracts, add architecture checks, add lockfiles, improve bootstrap, and improve release quality.

3. Architecture Inspiration

System	Useful lesson for Kiste
Go modules	Explicit module path, versioning, dependency graph, and lockable state.
Node/npm	Package metadata, scripts, dependency declaration, and ecosystem growth.
Python packaging	Installable packages, optional extras, console entry points, and importable APIs.
Terraform	Small core, provider/plugin boundary, clear external system integrations.

Kiste should learn from these systems without copying too much too early. v0.9.0 should prepare the foundation, not build a full marketplace.

4. Static Plugins First

Use static plugins before dynamic plugins. In v0.9.0, plugins should be known at install time, declared in code, loaded through a registry, and verified by architecture checks.

static plugins first
remote marketplace later

plugins are known at install time
plugins declare metadata
plugins declare commands
plugins declare dependencies
plugins declare capabilities
plugins are loaded only when enabled

Avoid in v0.9.0:

remote plugin download at runtime
untrusted plugin execution
automatic plugin update
marketplace install from unknown sources
dynamic code loading from GitHub

5. Three-Way Split-Ready Repository Design

Design the current repo as a monorepo with strict boundaries, so Kiste can later split into three repos without a rewrite. Do not split too early.

Future repos:

KisteBox/kiste-core -> engine, contracts, models, plugin system
KisteBox/kiste-py -> Python SDK / public wrapper
KisteBox/kiste-cli -> command-line interface

Future repo	Responsibility
kiste-core	Platform brain: models, schemas, plugin contracts, registries, workspace graph logic, reports, validation, lockfile, and architecture rules.
kiste-py	Public Python SDK and compatibility wrapper: <code>scan_repo()</code> , <code>load_workspace()</code> , <code>generate_gitops_repo()</code> , <code>check_token()</code> , <code>inspect_blob()</code> , etc.
kiste-cli	Terminal UX: parsing, help text, output rendering, JSON mode, exit codes, prompts, and command ergonomics.

Most important rule:

Core does not know CLI exists.
Python wrapper does not know CLI exists.
CLI depends on Python wrapper.

6. Current Monorepo Layout Before Splitting

For v0.9.0, keep one repository but organize it as if it could be split later.

kiste/
 pyproject.toml
 README.md

```

packages/
  kiste-core/
    pyproject.toml
    src/kiste_core/
      __init__.py
      models.py
      schemas.py
      config.py
      storage.py
      reports.py
      errors.py
      registry.py
      plugins.py
      lockfile.py
      architecture.py

  kiste-py/
    pyproject.toml
    src/kiste/
      __init__.py
      scanner.py
      workspace.py
      gitops.py
      secrets.py
      tokens.py
      data.py
      blob.py
      module.py

apps/
  kiste-cli/
    pyproject.toml
    src/kiste_cli/
      __init__.py
      main.py
      commands/
        scan.py
        workspace.py
        gitops.py
        secrets.py
        data.py
        blob.py
        module.py
        release.py
        architecture.py

plugins/
  kiste-repo/
  kiste-workspace/
  kiste-gitops/
  kiste-data/
  kiste-blob/
  kiste-argocd/
  kiste-cloud/

contracts/
  schemas/
    repo-spec.schema.json
    workspace-spec.schema.json
    dataset-spec.schema.json
    blob-ref.schema.json
    plugin.schema.json

tests/
  core/
  py/
  cli/
  integration/
  architecture/

```

7. Dependency Direction

The dependency graph must be one-way. This is what makes future splitting possible.

```

kiste_core
^
plugins depend on kiste_core
^
kiste-py depends on kiste_core + plugins

```

^
kiste-cli depends on kiste-py

Never allow:

```
kiste_core -> kiste
kiste_core -> kiste_cli
plugin -> kiste_cli
kiste-py -> kiste_cli
library API -> cli.py
```

Good example:

```
# kiste_cli/commands/scan.py
from kiste.scanner import scan_repo
```

Bad examples:

```
# kiste/scanner.py
from kiste_cli.main import scan_command

# kiste_core/models.py
from kiste.cli import something
```

8. Layer Ownership

8.1 kiste-core owns platform primitives

- RepoSpec, WorkspaceSpec, DatasetSpec, BlobRef, ImageSpec, TargetPlan, TokenMetadata, ReadinessReport.
- PluginSpec, ModuleManifest, KisteLock, CommandSpec, CapabilitySpec.
- Plugin API, module registry, schema versions, architecture checks, reports, graph primitives, lockfile, config loading, and storage abstraction.

Core should not own terminal formatting, print statements, argparse/typer commands, interactive prompts, or CLI UX.

8.2 kiste-py owns the public Python SDK

```
from kiste.scanner import scan_repo
from kiste.workspace import load_workspace, scan_workspace
from kiste.gitops import generate_gitops_repo, validate_gitops_repo
from kiste.tokens import check_token
from kiste.blob import inspect_blob, create_blob_manifest
```

The wrapper can be thin. It preserves user imports even when internal modules move.

```
# packages/kiste-py/src/kiste/scanner.py
from kiste_repo.scanner import scan_local_repo
```

```
def scan_repo(path: str = "."):
    return scan_local_repo(path)
```

8.3 kiste-cli owns command UX only

- Parse arguments.
- Call kiste-py functions.
- Format output as text, table, or JSON.
- Handle --verbose, --quiet, and --json.
- Return predictable exit codes.
- Show help text and examples.

```
# apps/kiste-cli/src/kiste_cli/commands/scan.py
from kiste.scanner import scan_repo
from kiste_cli.render import render_report
```

```
def scan_command(path: str = ".", json_output: bool = False):
    report = scan_repo(path)
    if json_output:
        print(report.model_dump_json(indent=2))
```

```

else:
    render_report(report)

```

The CLI should be disposable. If kiste-cli is deleted, the Python library should still work.

9. Static Plugin Contract

Each plugin should expose one static plugin object. It declares metadata, commands, capabilities, dependencies, and plugin API compatibility.

```

# plugins/kiste-repo/src/kiste_repo/plugin.py
from kiste_core.plugins import KistePlugin, CommandSpec, CapabilitySpec

PLUGIN = KistePlugin(
    name="repo",
    package="kiste_repo",
    version="0.9.0",
    api_version="0.1",
    commands=[
        CommandSpec(name="scan", capability="repo.scan"),
        CommandSpec(name="repo", capability="repo.inspect"),
    ],
    capabilities=[
        CapabilitySpec(name="repo.scan.local"),
        CapabilitySpec(name="repo.scan.github"),
    ],
    requires=[],
)

```

Static registration example:

```

from kiste_repo.plugin import PLUGIN as REPO_PLUGIN
from kiste_workspace.plugin import PLUGIN as WORKSPACE_PLUGIN
from kiste_gitops.plugin import PLUGIN as GITOPS_PLUGIN

STATIC_PLUGINS = [
    REPO_PLUGIN,
    WORKSPACE_PLUGIN,
    GITOPS_PLUGIN,
]

```

10. Module Manifest

Kiste should write and validate a local module manifest. This keeps module state explicit and helps bootstrap, doctor, release, and future package management.

```

{
  "kiste_version": "0.9.0",
  "plugin_api_version": "0.1",
  "modules": [
    {
      "name": "repo",
      "package": "kiste_repo",
      "version": "0.9.0",
      "enabled": true,
      "commands": ["scan", "repo"],
      "requires": []
    },
    {
      "name": "workspace",
      "package": "kiste_workspace",
      "version": "0.9.0",
      "enabled": true,
      "commands": ["workspace", "connect", "disconnect"],
      "requires": ["repo", "tokens"]
    }
  ]
}

```

File path:

.kiste/modules.json

11. Lockfile

Add kiste.lock as the compatibility anchor. It is not a full package manager yet. It records exact versions and schema/plugin compatibility.

```
{
  "kiste": "0.9.0",
  "plugin_api": "0.1",
  "schema": "0.9",
  "modules": {
    "repo": "0.9.0",
    "workspace": "0.9.0",
    "gitops": "0.9.0",
    "secrets": "0.9.0",
    "blob": "0.9.0"
  },
  "generated_at": "2026-05-27T00:00:00Z"
}
```

The lockfile should record:

- Kiste version.
- Plugin API version.
- Schema version.
- Enabled module versions.
- Provider versions.
- Generated file format versions.
- Release/build metadata when useful.

12. Package-Manager Vision Without Marketplace Yet

Kiste should prepare for package-manager behavior but not launch a marketplace in v0.9.0.

Future direction:

```
kiste install repo
kiste install data
kiste install aws
kiste install argocd
kiste module update
kiste module lock
kiste module verify
```

v0.9.0 prepares only:

```
module metadata
static plugin registry
lockfile
compatibility checks
module doctor
```

This lets Kiste grow like an ecosystem later, while staying safe and predictable now.

13. Commands for Phase 9

Command	Purpose
kiste module list	Show installed and enabled static modules.
kiste module graph	Show module dependency graph.
kiste module doctor	Check missing dependencies, disabled modules, and invalid metadata.
kiste module lock	Generate or update kiste.lock.
kiste module verify	Verify module versions and plugin API compatibility.
kiste architecture check	Check dependency boundaries and forbidden imports.
kiste architecture graph	Display core -> plugin -> py -> cli dependency direction.

Command	Purpose
kiste architecture doctor	Explain fixes for architecture violations.

Example architecture check output:

```
Kiste architecture check

[OK] CLI imports library modules
[FAIL] Library module imports cli.py
[OK] Static plugin registry found
[OK] modules.json exists
[OK] kiste.lock exists
[WARN] data module enabled but provider extras missing
[OK] no dynamic plugin download enabled
```

14. Refactor Target

Current bad direction:

```
kiste.cli owns scan logic
kiste.cli owns workspace logic
kiste.cli owns GitOps logic
kiste.cli owns secret logic
library wrappers import cli
```

Target direction:

```
kiste.cli imports kiste-py
kiste-py imports modules
modules own logic
kiste_core owns contracts, models, registries, and lockfile
reports own rendering data structures
CLI owns output formatting only
tests target modules first, CLI second
```

Final dependency direction:

```
kiste_core
^
modules depend on core
^
kiste-py depends on modules
^
cli depends on kiste-py
```

```
Never:
core -> cli
module -> cli
library API -> cli
```

15. API Freeze for v0.9.0

Keep these public imports stable:

```
from kiste.scanner import scan_repo
from kiste.workspace import load_workspace, scan_workspace
from kiste.gitops import generate_gitops_repo, validate_gitops_repo
from kiste.secrets import SopsSecretBackend
from kiste.tokens import check_token
from kiste.blob import inspect_blob, create_blob_manifest
```

Add stable internal platform APIs:

```
from kiste_core.plugins import KistePlugin
from kiste_core.registry import PluginRegistry
from kiste_core.models import RepoSpec, WorkspaceSpec, DatasetSpec, BlobRef
from kiste_core.lockfile import KisteLock
```

Import space	Meaning
kiste.*	User-facing Python SDK. Should remain stable for users.
kiste_core.*	Stable internal platform API for plugins and wrappers.

Import space	Meaning
kiste_repo.*, kiste_data.*, etc.	Module APIs. Can evolve, but should follow plugin API compatibility.
kiste_cli.*	CLI-only UX. No library/module should depend on it.

16. Package Names and Versioning

Distribution package	Import package	Purpose
kiste-core	kiste_core	Low-level engine and plugin contracts.
kiste	kiste	Python wrapper / SDK.
kiste-cli	kiste_cli	CLI executable and command UX.

Recommended version compatibility:

```
kiste-core 0.9.0
kiste-py   0.9.0
kiste-cli  0.9.0
```

Future:

```
kiste-cli 0.9.3 depends on kiste >=0.9,<1.0
kiste     0.9.3 depends on kiste-core >=0.9,<1.0
```

```
plugin_api = 0.1
schema_version = 0.9
```

17. Architecture Tests

Phase 9 tests should prove architecture quality, not new features.

```
tests/architecture/
  test_core_does_not_import_cli.py
  test_core_does_not_import_kiste_py.py
  test_python_wrapper_does_not_import_cli.py
  test_plugins_do_not_import_cli.py
  test_cli_only_imports_public_kiste_api.py
  test_public_api_imports_stable.py
  test_static_plugin_registry.py
  test_module_manifest.py
  test_module_lockfile.py
  test_plugin_api_compatibility.py
  test_bootstrap_minimal.py
```

Boundary test example:

```
from pathlib import Path

FORBIDDEN_IMPORTS = {
    "packages/kiste-core": ["kiste_cli", "from kiste ", "import kiste."],
    "packages/kiste-py": ["kiste_cli"],
    "plugins": ["kiste_cli"],
}

def test_layer_boundaries():
    for root, forbidden_items in FORBIDDEN_IMPORTS.items():
        for path in Path(root).rglob("*.py"):
            text = path.read_text(encoding="utf-8")
            for item in forbidden_items:
                assert item not in text, f"{path} illegally imports {item}"
```

18. Split Plan Later

Do not split immediately. Use staged separation so each step is testable.

Version	Goal
v0.9.0	Monorepo with hard boundaries, static plugins, manifest, and lockfile.
v0.9.1	Internal packages build separately inside the monorepo.

Version	Goal
v0.9.2	CLI package can be installed separately.
v0.9.3	kiste-core and kiste-py can be released separately if needed.
v1.0	Optional split into three repos when boundaries are proven and maintenance benefits are clear.

Split only when:

- Boundaries are already clean.
- Architecture tests prove no bad imports.
- Packages build independently.
- Users need separate install paths.
- Release automation can publish each piece reliably.

19. What Phase 9 Should Not Do

- No new provider families.
- No new cloud deployment engine.
- No new data platforms.
- No registry marketplace.
- No remote plugin installation.
- No automatic plugin update.
- No SaaS features.
- No runtime controller.

20. Acceptance Criteria

Phase 9 / v0.9.0 is complete when these pass:

```
kiste architecture check
kiste architecture graph
kiste architecture doctor
kiste module list
kiste module graph
kiste module doctor
kiste module lock
kiste module verify
kiste release check
pytest
```

Required outputs:

- [OK] static plugin registry.
- [OK] module manifest.
- [OK] kiste.lock.
- [OK] plugin API version.
- [OK] command registry.
- [OK] no library module imports cli.py.
- [OK] no core import depends on kiste-py or kiste-cli.
- [OK] public Python API still works.
- [OK] bootstrap core still works.
- [OK] release checks pass.

21. Final Direction

Phase 9 / v0.9.0 is the "make Kiste architecturally serious" release.

Design Kiste like this:

```
core = platform brain
python wrapper = developer API
cli = user interface
```

Final rule:

Kiste must be developed as one repo today, but with boundaries strict enough that kiste-core, kiste-py, and kiste-cli can become three repos later without architectural surgery.